TP1 Continuation Notes

Pull the repository, verify the baseline, continue from slide 6

Project: `todo-api` ▪ Current boundary: TypeORM configured

This handoff records exactly what is implemented and gives a safe continuation path. Do not start by rewriting the existing architecture. Run the baseline checks first, then extend it feature by feature.

Remember The repository is intentionally stopped before validation and Todo CRUD. Your first task is slide 6: DTO constraints and validation.

1 Clone and verify

After the repository is published, clone it with:

```
git clone https://github.com/MohamedYassineTurki/tp1-nestjs-todo-api.git
cd tp1-nestjs-todo-api
npm install
npm run build
```

Start development mode:

```
npm run start:dev
```

In another terminal, verify the completed endpoint:

```
curl http://localhost:3000/uuid-test
```

Expected response:

```
{"id":"a-generated-uuid"}
```

Repeat the request. The value should change.

2 Current structure

```
src/
  main.ts                          Application startup
  app.module.ts                    Root module and TypeORM connection
  app.controller.ts                Initial Hello World route
  app.service.ts                   Initial service
  common/
    common.module.ts               Global shared module
    constants/common.constants.ts UUID provider token
  uuid-test/
    uuid-test.module.ts            Test feature module
    uuid-test.controller.ts        GET /uuid-test
    uuid-test.service.ts           Injects and calls UUID provider
  todos/
    entities/todo.entity.ts        TypeORM Todo entity
    enums/status.enum.ts           Temporary three-value status enum
todo.sqlite                        Local development database
```

The project currently has no Todo controller or Todo service. The entity exists, but no Todo repository is injected yet.

3 Completed work

Nest application

`src/main.ts` bootstraps `AppModule` and listens on port 3000. `AppModule` registers the root controller, root service, common module, UUID test module, and TypeORM connection.

UUID provider

`CommonModule` imports `v4 as uuidv4` from `uuid`, registers it with `useValue`, and exports the symbol token `UUID_PROVIDER`. The global decorator makes the shared provider available to feature modules.

```
{
  provide: UUID_PROVIDER,
  useValue: uuidv4,
}
```

`UuidTestService` receives the function with `@Inject(UUID_PROVIDER)`. Its controller exposes `GET /uuid-test`.

Entity and database

`TodoEntity` contains an auto-generated integer ID, name, description, creation timestamp, and status. `AppModule` configures TypeORM with `better-sqlite3`, the file `todo.sqlite`, and `synchronize: true` for local learning. The earlier configuration `type: 'sqlite'` failed with the installed TypeORM definitions. The working driver name is `better-sqlite3`; both the package and the configuration are already present.

4 Your first continuation: slide 6

Install validation packages:

```
npm install class-validator class-transformer
```

Add the global validation pipe in `src/main.ts`:

```
import { ValidationPipe } from '@nestjs/common';

app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,
    forbidNonWhitelisted: true,
  }),
);
```

Create `src/todos/dto/create-todo.dto.ts`. The assignment requires a required description of at least 10 characters and a required name between 3 and 10 characters:

```
import { IsString, Length, MinLength } from 'class-validator';

export class CreateTodoDto {
  @IsString()
  @Length(3, 10)
  name!: string;

  @IsString()
  @MinLength(10)
  description!: string;
}
```

Create a separate update DTO with optional fields, retaining the size constraints. Add status validation with `IsEnum(StatusEnum)`. Put all custom error messages in one reusable constants file.

5 Remaining TP sequence

Follow this order so each step has a working foundation:

1. Slide 6: validation pipe, create DTO, update DTO, reusable messages, enum validation.
2. Slide 7: add `updatedAt` and `deletedAt`; keep `createdAt` controlled by TypeORM and excluded from update input.
3. Slide 8: extract creation, update, and deletion timestamp columns into a reusable base entity.
4. Slide 9: generate `TodosModule`, `TodosService`, and `TodosController`; import `TypeOrmModule.forFeature([TodoEntity])`; implement `addTodo` and preserve both requested versions.
5. Slide 10: implement update by ID and handle a missing Todo.
6. Slides 11-12: implement hard delete, soft delete, and restore.
7. Slide 13: add one aggregation endpoint returning counts for the three statuses.
8. Slides 14-15: get all Todos and get one Todo by ID with `NotFoundException`.
9. Slide 16: add optional search and status filters with the intended `(name OR description) AND status` logic.
10. Slide 17: add validated page and limit parameters, stable ordering, skip, take, and useful pagination metadata.

6 Recommended Todo wiring

When you create the feature module, the repository registration belongs in that module:

```
@Module({
  imports: [TypeOrmModule.forFeature([TodoEntity])],
  controllers: [TodosController],
  providers: [TodosService],
})
export class TodosModule {}
```

The service then receives the repository:

```
constructor(
  @InjectRepository(TodoEntity)
  private readonly todoRepository: Repository<TodoEntity>,
) {}
```

Keep HTTP extraction in the controller and database/application logic in the service.

7 Baseline issues to know

The application build passes and the UUID endpoint works. The CLI-generated unit tests for `UuidTestService` and `UuidTestController` currently fail because their isolated test modules do not provide the UUID token and service dependency. Before final delivery, update those tests with mocks or imported modules. Do not mistake a unit-test dependency error for a runtime application error. The local file `todo.sqlite` should not be committed. Add it to `.gitignore`; the database can be recreated from the entity and TypeORM configuration. Also ignore `node_modules`, `dist`, and `coverage`.

8 Git workflow

After each coherent TP milestone:

```
git status
git add src package.json package-lock.json .gitignore
git commit -m "Implement slide 6 validation"
git push
```

Do not commit passwords, database credentials, or generated dependency folders. Keep commits small enough that the professor can follow the architecture.